

# Primoji: Compositional Semantic Tokenization for Language Model Training

Frane Bandov  
offbyte  
frane@offbyte.com

## Abstract

We introduce Primoji, a new tokenizer for language models, that builds on semantic structure. Its vocabulary contains about ten thousand tokens organised into three semantic and one fallback tiers. Most words are either expressed directly as semantic primitives, e.g. **FIRE** for fire or as short sequences of primitives, e.g. photosynthesis becomes **[PLANT, HAVE, LIGHT]** or evaporation becomes **[WATER, CAUSE, AIR]**.

To test the viability of Primoji, we compared it to the state of art Mistral v0.3 BPE tokenizer (vocabulary size 32,768) on FineWeb-Edu. At 125M parameters and matched FLOPs, we find that Primoji reaches 1.080 bits-per-byte vs. BPE’s 1.144, a 5.5% gap, but at 1B the gap collapses. As a result we conclude that a vocabulary three times smaller than current ones can be built and is viable and enables new architectural ideas, which are not tested in this paper, but remain future work to be built on it.

## 1 Introduction

Tokenization for language model pretraining has mostly settled on Byte Pair Encoding since GPT-2 [17, 11], and most current models use BPE or a close variant. There are good reasons for this as BPE compresses well, the training is fast and deterministic, the algorithm has no hyperparameters beyond vocabulary size, and the resulting tokenizer handles arbitrary input through byte fallback. The tradeoff is that BPE has no semantic structure: *un* appears in *unhappy* and *university* with no compositional relationship the model can use, and Gallé [5] showed that most of BPE’s effectiveness comes from sequence-length compression rather than from any meaningful subword structure. The natural question for us was what happens if you build the structure in deliberately.

The concept is not new and as old as writing itself. Egyptian hieroglyphs used determinatives as semantic classifiers, Chinese builds *volcano* from *fire* plus *mountain* and *computer* from *electric* plus *brain*, while German speakers parse *Handschuh* (hand-shoe = glove) and immediately understand novel coinages like *Mondschuh* (moon-shoe). Wierzbicka [20] and Goddard and Wierzbicka [6] spent five decades arguing that 65 semantic primes, across more than 30 language families, suffice as a basis for defining word meanings, and the Longman Defining Vocabulary uses about two thousand words to define all 80,000+ entries in a full English dictionary. The natural question for neural models is what kind of impact the same structure has on their training and generalisation.

Primoji is a tokenizer built around Wierzbicka’s primes, with a vocabulary of about ten thousand tokens organised into four tiers. We trained 125M and 1B transformers on FineWeb-Edu with Primoji and with Mistral-v0.3 BPE to find out whether it actually works at training scale. We have found that the 125M Primoji reaches 1.080 bits-per-byte at matched FLOPs against BPE’s 1.144 and that at 1B both tokenizers reach the same bits-per-byte at matched FLOPs (Primoji 0.965

per document, BPE 0.985, equal once compute is held constant). Therefore, we conclude that the Primoji tokenizer works: The smaller vocabulary trains, learns, and produces coherent output at performance levels that are comparable to a standard tokenizer, but with trade-offs like rougher grammar and weaker creative and stylistic text.

However, the smaller vocabulary also opens architectural directions that are harder to explore with a 32K vocabulary. We also describe a secondary technique called compositional embedding aliases (Section 4.5), which came out of necessity rather than intention and applies independently of Primoji.

## 2 Related work

Tokenization for LLM pretraining has been dominated by frequency-driven approaches since GPT-2: BPE [17], WordPiece [23], Unigram LM [10], and SentencePiece [11] all pick subword units by statistical properties of the training corpus. The premise of these methods is that compression is what tokenization should optimise. Gallé [5] gives the empirical ground for questioning this: BPE’s compositional structure contributes only marginally to downstream effectiveness, with sequence-length compression doing most of the work. Primoji is built on the opposite premise. We test whether imposing semantic structure at the input, even at the cost of some compression, gives the model useful signal it otherwise has to reconstruct.

The closest active research asking similar questions is morphological tokenization. MorphPiece [8] and MorphBPE [1] build vocabularies along word-formation lines, splitting *running* into *run* plus *ning*, and both report gains at scale against BPE baselines. The structural commitment is the same as ours, impose linguistic structure on the vocabulary rather than letting frequency alone decide. The structural choice is different: morphological structure is a property of the language, fixed by how English (or any target language) happens to build words. Semantic structure as we use it is conceptual, grounded in NSM primitives that Wierzbicka and colleagues argue hold across languages, and adaptable to domains by swapping the dictionary. Neither literature has compared the two structure types directly under matched conditions, and we expect the answer depends on what the model is being trained for.

A complementary line changes the training objective rather than the tokens. Iyer et al. [7] keep BPE tokens but predict at concept level, reporting lower perplexity and better robustness to domain shift. Primoji keeps next-token prediction but changes what a token is. The two approaches attack the same underlying problem (LLMs spend capacity recovering conceptual structure the input doesn’t provide) from different directions, and are in principle composable.

Pagnoni et al. [13] take the opposite extreme with byte-level transformers and dynamic entropy-based patching, matching BPE at 8B. Their patches are statistical and emerge during training; ours are semantic and specified by dictionary. The deep question both approaches raise is whether tokens should be fixed at all or dynamic, and we do not take a position on this here.

Two pieces of evidence support the design premise. Wies et al. [21] show that when vocabulary size is small relative to model width, additional width gives diminishing returns, which is consistent with our 1B finding that Primoji’s parameter savings help less at scale. Kaplan et al. [9] find that LLMs spend early and middle layer capacity reconstructing whole-word representations from BPE subwords, suggesting the unstructured input costs real model capacity. Primoji is one way to provide that structure at the input directly, though whether models trained on Primoji spend less capacity on lexical reconstruction is something we have not measured.

The NSM framework we use comes from Wierzbicka [20] and Goddard and Wierzbicka [6], who argue that a small set of semantic primes (around 65) suffices as a reductive base for meaning

across languages. NSM has been a linguistic theory for decades with little computational uptake until recently. DeepNSM [2] fine-tunes LLMs to generate NSM explications for arbitrary words, using NSM as an output target. Primoji uses NSM as an input vocabulary. The two directions are complementary: DeepNSM-style models could automate dictionary construction for new Primoji domains, which is currently the bottleneck in building a domain-specific variant. Ball et al. [3] anchor word2vec dimensions to NSM primitives for use in the ACT-R cognitive architecture, a post-hoc structuring of existing embeddings. Primoji moves this structuring to pretraining time so the primitives get their own learned embeddings in a language model rather than fixed projections of distributional vectors.

Our compositional embedding aliases draw on a separate line about structured embeddings. Reif et al. [15] replace surface-form embeddings with compositions of base-form embeddings and learned morphological transformation vectors, so *running* is built from *run* plus a progressive-aspect vector. Our aliases do the same kind of thing with semantic rather than morphological components: *is* and *was* share a **BE** component, *was* and *before* share a **BEFORE** component. The mechanism is independent of Primoji and applies to any tokenizer with a meaningful decomposition.

Compositional semantic structure in human writing systems long predates computational work. Egyptian determinatives classified words by semantic category (fire, water, divinity, motion), Chinese characters combine radicals compositionally (volcano = fire + mountain, computer = electric + brain), and German productively compounds nouns in ways speakers parse on the fly (*Handschuh*, hand-shoe = glove). These are not neural tokenizers, but they are the historical evidence that humans build meaning by composition and can generalise productively from small primitive sets to large lexicons. The design premise of Primoji rests on this prior about how meaning-making works.

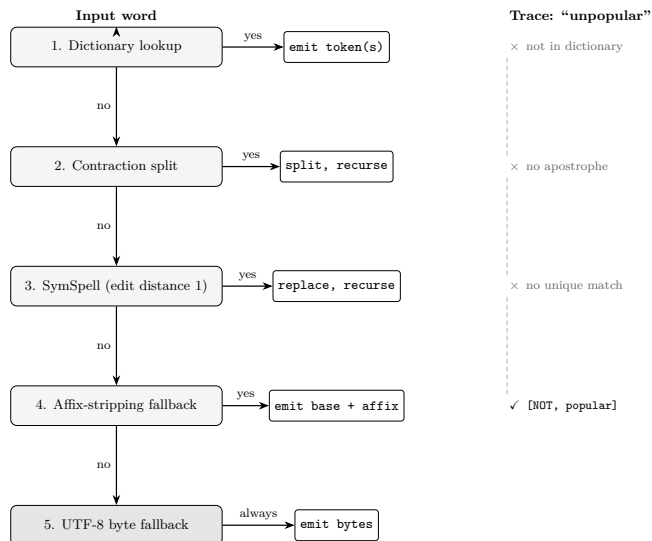
### 3 The Primoji vocabulary and pipeline

The Primoji vocabulary is a fixed set of about ten thousand tokens organised into four tiers: emoji, NSM primitives, common English words plus structural tokens, and a UTF-8 byte fallback. The four tiers are ordered by encoding precedence. Most complex words are pre-decomposed into primitive sequences at build time and stored in the dictionary, so *photosynthesis* looks up directly to [PLANT, HAVE, LIGHT]. A small runtime fallback handles the rare OOV cases via affix stripping. Anything still unresolved falls through to UTF-8 bytes. The full pipeline is shown in Figure 1 and described in Section 3.2.

We built Primoji to find out what happens if you construct a vocabulary out of semantic primitives instead of statistical merges. Work like MorphPiece [8] and MorphBPE [1], which builds vocabularies along morphological lines (*run* + *ning*) and reports gains at 300M and 1B, encouraged us that thinking differently about tokenization is worth the effort.

We wanted to know three things: is the approach feasible to construct at all, if a model trained on it actually learns and produces coherent output, and if the BPB it reaches is comparable to what a standard tokenizer reaches at the same compute. We use BPE as the baseline for the third question because it is the standard.

We use Wierzbicka’s NSM primes as the primitive set because NSM is the most developed attempt at identifying a universal set of semantic primitives. While NSM is a pragmatic starting choice, we do not claim it is the only or best choice. The design freedom we expose matters more than the specific primitive set, and the same architecture would accept a different set with different tradeoffs.



**Figure 1:** The Primoji encoding pipeline, simplified view. Each input word descends through five stages, stopping at the first that returns a result; the full code path has additional pre-processing steps (for punctuation, numbers, and contractions) not shown here. The trace shows *unpopular*, which is not in the dictionary and resolves at stage 4 via affix stripping to [NOT, popular]. Words whose compositions are pre-stored in the dictionary (such as *photosynthesis* = [PLANT, HAVE, LIGHT]) resolve at stage 1, not at stage 4.

### 3.1 Vocabulary tiers

The vocabulary has four tiers with strict encoding precedence. The numbers below are for V6, the sixth iteration and version used in the main experiments.

**Tier 1a, direct emoji (1,175 tokens).** All Unicode emoji after deduplication. The variation selectors (U+FE0F) are stripped before duplicate comparison, which removed 18 hidden collisions earlier versions had missed. Each emoji maps to one or more English words via the dictionary.

**Tier 1b, common word tokens (~7,800 tokens).** High-frequency English words from the New General Service List and from FineWeb-Edu corpus statistics. The initial vocabulary had only 1,276 words here, which forced too many common terms through the byte fallback. We expanded the tier to 7,800 words across seven revisions as we tracked down which gaps mattered (Section 4.2).

**Tier 2, compositional primitives (140 tokens).** The 65 NSM primes [6] plus 75 domain expansions: perceptual primitives (HOT, COLD, HEAVY, LINE) following NSM’s tradition of perceptual molecules, grammatical markers (WITH, FOR, ABOUT), and concrete domain terms (ENERGY, COLOR, ELECTRIC, LOVE, FEAR).

**Tier 3, structural and geographic (~830 tokens).** 249 country flags as geographic modifiers, ~500 proper-noun anchors extracted with spaCy NER from 100K FineWeb-Edu documents, and ~84 structural tokens (punctuation, math symbols, currency, Greek letters).

**Tier 4, byte fallback (258 tokens).** Following the LLaMA tokenizer, anything not captured by the previous tiers falls through to individual UTF-8 bytes.

**A note on the name.** Primoji combines *primitives* and *emoji*, but the model never sees emoji glyphs, only integer IDs. The emoji tier could be replaced with word tokens without changing the architecture or the results. When the model generates an emoji ID we decode it to the corresponding English word.

## 3.2 Encoding pipeline

The pipeline is shown in Figure 1: each input word runs through five stages in order (exact lookup, contraction split, spell correction, affix fallback, byte fallback), stopping at the first that resolves it. Additionally, there are a few implementation details worth explaining:

**Contraction split.** Words like `don't` or `won't` split into `do + not` and `will + not` respectively. Unapostrophised forms are not handled as contractions; `dont` resolves through the word-token tier because it happens to be common enough to appear there, but `wont` and `cant` fall through to byte fallback.

**Spell correction.** Misspellings can bypass the dictionary even when the intended word is there, which wastes tokens on byte fallback. We use SymSpell at edit distance 1, accepting only unique candidates of four characters or more. Looser settings (e.g. edit distance 2) led to more false corrections on our test sample and hurt more than they helped.

**Compositional entries.** Most complex words are decomposed into primitives at build time and stored directly in the dictionary (about 9,000 such pre-computed compositions). At runtime, words that are still not in the dictionary fall through to a small affix-stripping fallback that handles about 0.8% of input words: `unpopular` becomes `[NOT, popular]`, for example. The affix fallback is limited in what it can do correctly, and many edge cases fall through to byte fallback instead.

**Dictionary sources.** The encoding dictionary was built by layering five public resources: Unicode CLDR v48 annotations, the emojiLib community keyword set, EmojiNet [22], emoji2vec [4], and ELCo [24]. Around 42,000 entries total: 26,434 word tokens, 9,081 pre-composed primitive sequences, 3,869 emoji mappings, and 2,833 proper-noun anchors. The build is reproducible from a single script (`scripts/build_dictionary.py`) so that vocabulary re-sorts do not invalidate seeded mappings.

**Decoding.** The decoder uses the vocabulary directly (catalog, then primitives, then anchors, then bytes) and never reverses through the encoding dictionary. We tried the reverse path initially and it created ambiguity: multiple words map to the same composition (`[FAST, SOMEONE]` could be *runner* or *sprinter*). For the canonical decode of a known composition the priority is single-word CLDR name, then corpus frequency via `wordfreq`, then shortest form.

**Math and code.** Numbers and code do not compose through semantic primitives. Primoji uses single-digit tokenization for digits, following Singh and Strouse [18], atomic tokens for math operators (`+`, `-`, `×`, `÷`, `=`, `≤`, `π`, `Σ`, `f`), and dedicated tokens for common code keywords. Composition through primitives would not work for these domains, so we do not attempt it.

## 4 Experiments

### 4.1 Setup

We trained two Primoji models, 125M and 1B parameters, on FineWeb-Edu [14]. For comparison, we trained the same architectures with Mistral v0.3 BPE (vocabulary size 32,768). Everything except the vocabulary is held constant. We measure bits-per-byte on a held-out validation split. Hyperparameters in Table 1.

**Table 1:** Training configuration. The 125M and 1B columns differ only in architecture size, learning rate, and grad-accumulation strategy. Both Primoji and BPE runs use identical configurations within each size.

|                          | 125M                      | 1B                        |
|--------------------------|---------------------------|---------------------------|
| Layers                   | 12                        | 24                        |
| Hidden dim               | 768                       | 2048                      |
| Attention heads          | 12                        | 16                        |
| FFN dim                  | 3072                      | 5461                      |
| Activation               | SwiGLU                    | SwiGLU                    |
| Normalization            | RMSNorm                   | RMSNorm                   |
| Sequence length          | 1024                      | 1024                      |
| Effective batch (tokens) | $32 \times 1024 = 32,768$ | $32 \times 1024 = 32,768$ |
| Micro batch / grad accum | $32 \times 1$             | $4 \times 8$              |
| Optimizer                | AdamW                     | AdamW                     |
| $\beta_1, \beta_2$       | 0.9, 0.95                 | 0.9, 0.95                 |
| Weight decay             | 0.1                       | 0.1                       |
| Gradient clip            | 1.0                       | 1.0                       |
| Peak LR                  | $6 \times 10^{-4}$        | $3 \times 10^{-4}$        |
| Min LR                   | $6 \times 10^{-5}$        | $3 \times 10^{-5}$        |
| LR schedule              | cosine                    | cosine                    |
| Warmup steps             | 2,000                     | 2,000                     |
| Mixed precision          | bf16                      | bf16                      |
| Eval interval            | every 200 steps           | every 500 steps           |

We report bits-per-byte (BPB), following Pagnoni et al. [13]. BPB normalizes loss by the original byte count and is therefore independent of vocabulary size and sequence length:

$$\text{BPB} = \frac{\sum \text{CE loss (nats)}}{\text{original text bytes} \times \ln 2}.$$

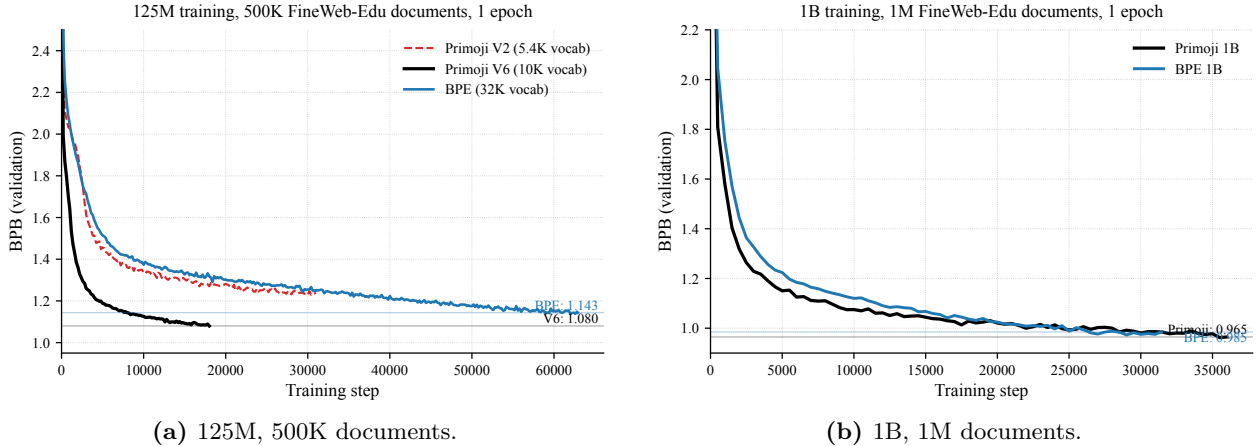
### 4.2 Compression: a vocabulary-gap story

The early Primoji vocabularies produced far more tokens per document than Mistral BPE. The initial version (V1, 4,035 tokens) produced  $1.79\times$  as many on a 1,000-sentence FineWeb-Edu sample. While we initially read this as a fundamental limitation of compositional tokenization, the cost of building meanings from a small primitive set, it turned out to be a vocabulary gap.

First improvements came from adding pre-computed compositions to the dictionary, as many complex words were falling through to byte fallback (8–10 tokens each), but could be stored as 2–3 primitives instead. Adding around 9,000 pre-computed entries dropped the ratio to  $1.53\times$  in the fourth iteration (V4). The second improvement was extracting high-frequency words from

**Table 2:** Compression progression across vocabulary versions on 1,000 FineWeb-Edu sentences.

| Version       | Vocab  | Tokens (Primoji) | Ratio vs BPE                    | Byte fallback |
|---------------|--------|------------------|---------------------------------|---------------|
| v1            | 4,035  | 59,460           | $1.795\times$                   | 17.4%         |
| v2            | 4,081  | 58,872           | $1.778\times$                   | 16.6%         |
| v3            | 5,311  | 57,120           | $1.725\times$                   | 16.6%         |
| v4            | 18,464 | 50,587           | $1.527\times$                   | 13.6%         |
| v6            | 10,195 | 36,076           | <b><math>1.089\times</math></b> | <b>6.9%</b>   |
| BPE (Mistral) | 32,768 | 33,118           | $1.000\times$                   | 0%            |

**Figure 2:** Validation BPB during training, one epoch. (a) At 125M, Primoji V6 (10K vocab) ends at 1.080, BPE (32K vocab) at 1.144. Primoji V2 (5.4K vocab) is shown for reference. (b) At 1B, Primoji ends at 0.965, BPE at 0.985. The gap narrows from 5.5% to 2%.

FineWeb-Edu itself, like *business*, *government*, *important*, *however*, and adding the top 5,000 as direct word tokens. We also split hyphenated forms (*insulin-deprived* into *insulin* plus *deprived*). The ratio dropped to  $1.09\times$  at V6 (10,195 tokens). A median sentence at the current Primoji iteration (V6) reaches exact BPE parity. The compositions account for about 7.8% of emitted tokens, concentrated on the rare and technical long tail.

The lesson learned between the iterations is that what actually seemed an algorithmic problem needed in fact a much simpler solution: adding more words to the word tier.

### 4.3 125M training

We trained Primoji and BPE at 125M parameters on 500K FineWeb-Edu documents for one epoch each. Primoji V6 reaches 1.080 BPB. Mistral BPE reaches 1.144. The 5.5% advantage showed up around step 3,000 (5% of training) and never reversed within the epoch.

Because the two tokenizers produce different token counts for the same documents, Primoji and BPE do not use the same amount of compute per epoch. Primoji’s sequences are about 9% longer on average, so one epoch over 500K documents is around 590M tokens for Primoji and 515M for BPE. The token difference is offset by a parameter difference in the other direction: Primoji has a smaller vocabulary and therefore a smaller embedding table (6% of the model vs 18% for BPE), leaving Primoji with 122M parameters against BPE’s 139M. Training FLOPs approximate as  $6 \times \text{params} \times \text{tokens}$ , which works out to roughly  $4.3 \times 10^{17}$  for both runs. The 5.5% BPB advantage



for Primoji is therefore at equal compute, not just equal data.

There is also a parameter-allocation effect that shrinks with scale. At 125M, BPE spends 18% of its parameter budget on the embedding table; Primoji spends 6%. The 17M parameters Primoji saves on embeddings get reallocated to attention and FFN, where they do useful work. At 1B the gap between the two embedding tables is only 47M parameters, or 3% of the total. As we will see in Section 4.6, the BPB advantage shrinks with it.

#### 4.4 The byte-preference problem

V1 at 125M generated 95% bytes and the model was spelling words out character by character instead of using primitives. We initially assumed it was a structural problem with compositional vocabularies and tried three fixes: pre-computed compositions, tier embeddings, and downweighting the loss on byte tokens by 0.7. Byte fallback went from 16.6% to 10.5% in training, but generation was still mostly byte-based.

The simple solution was adding more words, and V6 generates only 5.9% bytes. The primitives were not the problem, just the vocabulary was too small. Once *government* was a token the model used it.

#### 4.5 Compositional embedding aliases

The V5 version of Primoji maps grammar words like *is*, *not*, *this* straight to NSM primitives (BE, NOT, THIS) the way NSM theory says it should and this resulted in a BPB of 1.073, the lowest of anything we trained. However, the output was unreadable, as one would read output like BE NOT GOOD instead of *is not good*.

Our attempt to keep the compositional structure without the unreadable surface is aliases. An alias keeps *is* as a single token in the sequence but computes its embedding as the mean of its primitives:

```
embedding("is") = mean(embedding(BE), embedding(NOW))
embedding("was") = mean(embedding(BE), embedding(BEFORE))
embedding("they") = mean(embedding(SOMEONE), embedding(OTHER), embedding(MANY))
```

The alias has no parameters of its own. Updating [BE] updates *is*, *was*, *are*, *were*, *been*, *being* at once.

While we thought that this approach would help compositional generalization, it didn't, at least at one epoch. Adding 80 aliases moved BPB from 1.073 to 1.080, which is well within noise, but made the output more readable.

The closest prior work is VocabDiet [15], discussed in Section 2.

#### 4.6 Scaling to 1B

We also trained Primoji and BPE at 1B parameters on 1M FineWeb-Edu documents for one epoch. Primoji reaches 0.965 BPB with 1.18B tokens and  $9.03 \times 10^{18}$  FLOPs, while BPE reaches 0.985 BPB with 1.03B tokens and  $8.18 \times 10^{18}$  FLOPs. The two tokenizers end up in roughly the same place on both axes, and small differences of a few percent in BPB or compute could matter more or less depending on what you do with the model.

The 5.5% BPB gap established at 125M does not hold at 1B scale. The embedding savings that freed up parameter room are only 3% of the model at 1B instead of 14%, and a 1B model has enough capacity to extract structure implicit in BPE tokens on its own.



**Table 3:** Variant sweep at 125M, one epoch on 500K FineWeb-Edu documents. “Byte wt” is the loss-weight multiplier on byte-fallback tokens, “Tier emb” is whether the model receives a learned 5-way tier-type embedding, “Aliases” is the count of grammar words whose embeddings are composed from primitive components.

| Variant       | Total vocab | Byte wt | Tier emb | Aliases | BPB          | vs BPE       |
|---------------|-------------|---------|----------|---------|--------------|--------------|
| V1            | 4,035       | 1.0     | no       | 0       | 1.085        | −5.1%        |
| V2            | 4,081       | 0.3     | yes      | 0       | 1.096        | −4.2%        |
| V3            | 5,311       | 0.7     | yes      | 0       | 1.092        | −4.5%        |
| V4            | 18,464      | 0.7     | yes      | 0       | 1.085        | −5.1%        |
| V5            | 10,102      | 0.7     | yes      | 0       | 1.073        | −6.2%        |
| V6            | 10,195      | 0.7     | yes      | 80      | <b>1.080</b> | <b>−5.5%</b> |
| BPE (Mistral) | 32,768      | n/a     | n/a      | n/a     | 1.144        | baseline     |

**Table 4:** 1B comparison at one epoch on 1M FineWeb-Edu documents.

|                     | Primoji 1B            | BPE 1B                |
|---------------------|-----------------------|-----------------------|
| BPB                 | <b>0.965</b>          | 0.985                 |
| Parameters          | 1.231B                | 1.277B                |
| Tokens (1 epoch)    | 1.18B                 | 1.03B                 |
| FLOPs (1 epoch)     | $9.03 \times 10^{18}$ | $8.18 \times 10^{18}$ |
| Wall time (1 epoch) | 26.7 h                | 21.4 h                |

We also ran two architectural variants that the smaller vocabulary allows, namely a deeper-and-narrower (73 layers, 1024 hidden) and FFN-heavy variant. Both gained around 1.5%. The moves that would actually exploit a small vocabulary, hierarchical processing through composition boundaries, vocabulary-quadratic attention, MoE with per-expert dictionaries, are directions for future work.

## 4.7 Generation behavior

We ran a generation test on the V6 125M checkpoint with 200 prompts across five categories (science, history, ethics, technology, everyday) and asked the model to continue freely from each prompt. The question was what kinds of tokens the model produces in unrestricted generation: does it use primitives, or does it collapse to byte fallback? We classified each output token by tier and grouped multi-token byte sequences into single semantic units so byte fallback doesn’t get inflated by its 8–10 tokens per word. The output breaks down as 54.6% word tokens, 19.2% structural tokens, 13.7% primitives, 6.6% emoji, and 5.9% byte fallback (Table 5).

Three samples to convey what the output actually looks like:

*photosynthesis is process which plants convert light energy*  
*earthquake is caused move materials earth*  
*democracy is system which means power, consent, decency*

The output is compositional by construction: *earthquake* is stored in the vocabulary as three primitive tokens [EARTH, MOVE, DESTROY], not as one opaque word, so the model predicts those three tokens in sequence. The same holds for the thousands of complex words stored in the dictionary

**Table 5:** Tier distribution of generated tokens for V6 125M, 200 prompts. Confidence intervals are 95% binomial.

| Tier          | Semantic units | Percent | 95% CI      |
|---------------|----------------|---------|-------------|
| Word          | 7,339          | 54.6%   | $\pm 6.7\%$ |
| Structural    | 2,580          | 19.2%   | $\pm 5.3\%$ |
| Primitive     | 1,849          | 13.7%   | $\pm 4.6\%$ |
| Emoji         | 885            | 6.6%    | $\pm 3.3\%$ |
| Byte fallback | 799            | 5.9%    | $\pm 3.2\%$ |

as primitive sequences. Downstream benchmarks (MMLU, HellaSwag, math, code) need a larger and longer-trained Primoji and are left for future work.

## 5 Discussion

Primoji is a tokenizer built from semantic structure rather than statistical merges, and the main claim our experiments support is that it works. A vocabulary of semantic primitives plus a dictionary of compositions trains, learns, and generates coherent output at the scales we tested. The vocabulary is three times smaller than a standard BPE vocabulary and the model still trains.

One property of Primoji worth naming is interpretability. A Primoji trace shows which primitives the model sees, so *photosynthesis* appears as [PLANT, HAVE, LIGHT] rather than as a subword chunk like *photo* + *synth* + *esis*, and a reader inspecting the token stream can see what concepts are being expressed. The vocabulary is also inspectable and editable: each dictionary entry is a decision a human made and can revise. Whether these properties translate into practical value for debugging or model analysis is something we have not tested.

The most immediate practical use is domain specialisation: building a Primoji for medicine, law, or a scientific field means swapping the dictionary for one grounded in that domain’s concepts while keeping NSM primitives as the shared base, and the architecture and training pipeline do not change. Currently no other tokenizer enables this. Morphological tokenizers are fixed by the language, and frequency-driven ones are fixed by the corpus. Primoji’s dictionary is the component that encodes the domain, and swapping it is a few thousand lines of structured input rather than a retraining pass. The cost is that building the dictionary requires understanding the domain’s conceptual structure. For domains where the structure is already worked out, like medical ontologies or legal terminology, it is a capability the alternatives do not offer, but for general text there is an implied cost.

Primoji has several design parameters a user can set. Composition depth controls how many primitives can compose into one concept and trades coarse semantic groupings against sequence length. The primitive set controls cross-domain breadth against in-domain precision. The composition-rate dial controls how much of the training signal comes through primitives versus direct word tokens. Each of these is a real design decision a practitioner would need to understand before building a domain-specific Primoji.

Still, Primoji is not the right choice everywhere. Compositional decomposition is lossy at the surface form, so creative writing and stylistic text come out weaker, and code and math do not decompose through semantic primitives, which is why we handle them with dedicated tokens. Named-entity recall needs exact tokens, and rare entities fall through to byte fallback. FineWeb-Edu is close to the best case because educational text has a stable and well-defined vocabulary, and

other domains, especially conversational text, social media, and code, will look different.

Currently, our evaluation has several gaps. The dictionary and all experiments are English-only, we ran a single seed so the within-noise differences between variants should not be over-read, and we report BPB alone without downstream benchmarks. Downstream evaluation needs a larger and longer-trained Primoji than what we ran, which is why we did not attempt it here, but it is the most important gap to close. Multilingual evaluation is natural given NSM’s cross-linguistic foundation, though we have not tested it.

The natural next steps are architectures that exploit the smaller vocabulary, especially MoE with per-expert dictionaries where each expert has its own semantic specialization and the tokenization itself is the routing mechanism (building on Hash Layers [16] for deterministic routing and BitNet [19, 12] for sub-expert memory footprint). Other directions are hierarchical processing through composition boundaries, FFN-heavy reallocation of the embedding savings, extended 1B training with a downstream benchmark suite, and automated dictionary construction via NSM explication [2] to make domain specialisation tractable without manual effort.

## 6 Conclusion

A tokenizer with a 10K vocabulary of semantic primitives and pre-composed primitive sequences reaches BPB comparable to a 32K BPE vocabulary at 125M and 1B, trained on FineWeb-Edu at matched compute, and is interpretable, editable, and adaptable to new domains: swap the dictionary and the same architecture becomes a medical or legal Primoji.

What Primoji offers is a different approach to what tokenization can be. Frequency-driven tokenizers assume language is best chunked by statistical regularity. Primoji chunks it by semantic composition. Both can work, but the semantic version provides levers the statistical one does not, and our results say it does not cost anything at training time.

The immediate practical use is domain specialisation: swapping the dictionary for medicine, law, or a scientific field produces a different Primoji without changing the architecture. No other tokenizer offers this.

Whether the semantic vocabulary matters beyond viability is a question our results do not answer. The architectural directions it opens, hierarchical processing through composition boundaries, MoE with per-expert dictionaries, and vocabulary-quadratic attention, are future work.

## Code and Data Availability

Source code for the Primoji tokenizer, the dictionary build pipeline, the training scripts, and the evaluation scripts is available at <https://github.com/frane/primoji>. All training runs use FineWeb-Edu [14], a publicly available dataset. Total experiment cost: under \$200 on consumer and cloud GPUs.

## References

- [1] Ehsaneddin Asgari, Yassine El Kheir, and Mohammad Ali Sadraei Javaheri. MorphBPE: A Morpho-Aware Tokenizer Bridging Linguistic Complexity for Efficient LLM Training Across Morphologies, February 2025. URL <http://arxiv.org/abs/2502.00894>. arXiv:2502.00894 [cs].

- [2] Raymond Baartmans, Matthew Raffel, Rahul Vikram, Aiden Deringer, and Lizhong Chen. Towards Universal Semantics With Large Language Models, 2025. URL <https://arxiv.org/abs/2505.11764>. Version Number: 3.
- [3] Jerry T. Ball, Stuart Rodgers, and Alaric Ball. Creating meaningful word vectors and examining their use as representations of word meaning. 2024. URL <https://escholarship.org/uc/item/0bw8b0pf>. Preprint.
- [4] Ben Eisner, Tim Rocktäschel, Isabelle Augenstein, Matko Bosnjak, and Sebastian Riedel. emoji2vec: Learning Emoji Representations from their Description. In *Proceedings of The Fourth International Workshop on Natural Language Processing for Social Media*, pages 48–54, Austin, TX, USA, 2016. Association for Computational Linguistics. doi: 10.18653/v1/W16-6208. URL <http://aclweb.org/anthology/W16-6208>.
- [5] Matthias Gallé. Investigating the Effectiveness of BPE: The Power of Shorter Sequences. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 1375–1381, Hong Kong, China, 2019. Association for Computational Linguistics. doi: 10.18653/v1/D19-1141. URL <https://www.aclweb.org/anthology/D19-1141>.
- [6] Cliff Goddard and Anna Wierzbicka. *Words and meanings: lexical semantics across domains, languages, and cultures*. Oxford linguistics. Oxford University Press, Oxford New York, NY, first published in paperback edition, 2016. ISBN 978-0-19-966843-4 978-0-19-878355-8.
- [7] Laya Iyer, Pranav Somani, Alice Guo, Dan Jurafsky, and Chen Shani. Beyond Tokens: Concept-Level Training Objectives for LLMs, 2026. URL <https://arxiv.org/abs/2601.11791>. Version Number: 2.
- [8] Haris Jabbar. MorphPiece : A Linguistic Tokenizer for Large Language Models, 2023. URL <https://arxiv.org/abs/2307.07262>. Version Number: 2.
- [9] Guy Kaplan, Matanel Oren, Yuval Reif, and Roy Schwartz. From Tokens to Words: On the Inner Lexicon of LLMs. In *International Conference on Learning Representations (ICLR)*, 2025. URL <https://arxiv.org/abs/2410.05864>.
- [10] Taku Kudo. Subword Regularization: Improving Neural Network Translation Models with Multiple Subword Candidates. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 66–75, Melbourne, Australia, 2018. Association for Computational Linguistics. doi: 10.18653/v1/P18-1007. URL <http://aclweb.org/anthology/P18-1007>.
- [11] Taku Kudo and John Richardson. SentencePiece: A simple and language independent subword tokenizer and detokenizer for Neural Text Processing. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 66–71, Brussels, Belgium, 2018. Association for Computational Linguistics. doi: 10.18653/v1/D18-2012. URL <http://aclweb.org/anthology/D18-2012>.
- [12] Shuming Ma, Hongyu Wang, Lingxiao Ma, Lei Wang, Wenhui Wang, Shaohan Huang, Li Dong Ruiping Wang, Jilong Xue, and Furu Wei. The Era of 1-bit LLMs: All Large Language Models are in 1.58 Bits, 2024. Version Number: 1.

- [13] Artidoro Pagnoni, Ram Pasunuru, Pedro Rodriguez, John Nguyen, Benjamin Muller, Margaret Li, Chunting Zhou, Lili Yu, Jason Weston, Luke Zettlemoyer, Gargi Ghosh, Mike Lewis, Ari Holtzman, and Srinivasan Iyer. Byte Latent Transformer: Patches Scale Better Than Tokens, 2024. URL <https://arxiv.org/abs/2412.09871>. Version Number: 1.
- [14] Guilherme Penedo, Hynek Kydlíček, Loubna Ben allal, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro Von Werra, and Thomas Wolf. The FineWeb Datasets: Decanting the Web for the Finest Text Data at Scale, October 2024. URL <http://arxiv.org/abs/2406.17557>. arXiv:2406.17557 [cs].
- [15] Yuval Reif, Guy Kaplan, and Roy Schwartz. Vocab Diet: Reshaping the Vocabulary of LLMs with Vector Arithmetic, 2025. URL <https://arxiv.org/abs/2510.17001>. Version Number: 1.
- [16] Stephen Roller, Sainbayar Sukhbaatar, Arthur Szlam, and Jason Weston. Hash Layers For Large Sparse Models, 2021. URL <https://arxiv.org/abs/2106.04426>. Version Number: 3.
- [17] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural Machine Translation of Rare Words with Subword Units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1715–1725, Berlin, Germany, 2016. Association for Computational Linguistics. doi: 10.18653/v1/P16-1162. URL <http://aclweb.org/anthology/P16-1162>.
- [18] Aaditya K Singh and DJ Strouse. Tokenization counts: the impact of tokenization on arithmetic in frontier LLMs, 2024. URL <https://arxiv.org/abs/2402.14903>. Version Number: 1.
- [19] Hongyu Wang, Shuming Ma, Li Dong, Shaohan Huang, Huaijie Wang, Lingxiao Ma, Fan Yang, Ruiping Wang, Yi Wu, and Furu Wei. BitNet: Scaling 1-bit Transformers for Large Language Models, 2023. URL <https://arxiv.org/abs/2310.11453>. Version Number: 1.
- [20] Anna Wierzbicka. *Semantic primitives*. Number 22 in Linguistische Forschungen. Athenäum-Verl, Frankfurt a. M, 1972. ISBN 978-3-7610-4822-1.
- [21] Noam Wies, Yoav Levine, Daniel Jannai, and Amnon Shashua. Which Transformer architecture fits my data? A vocabulary bottleneck in self-attention, June 2021. URL <https://arxiv.org/abs/2105.03928>.
- [22] Sanjaya Wijeratne, Lakshika Balasuriya, Amit Sheth, and Derek Doran. A semantics-based measure of emoji similarity. In *Proceedings of the International Conference on Web Intelligence*, pages 646–653, Leipzig Germany, August 2017. ACM. ISBN 978-1-4503-4951-2. doi: 10.1145/3106426.3106490. URL <https://dl.acm.org/doi/10.1145/3106426.3106490>.
- [23] Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V. Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, Jeff Klingner, Apurva Shah, Melvin Johnson, Xiaobing Liu, Łukasz Kaiser, Stephan Gouws, Yoshikiyo Kato, Taku Kudo, Hideto Kazawa, Keith Stevens, George Kurian, Nishant Patil, Wei Wang, Cliff Young, Jason Smith, Jason Riesa, Alex Rudnick, Oriol Vinyals, Greg Corrado, Macduff Hughes, and Jeffrey Dean. Google’s Neural Machine Translation System: Bridging the Gap between Human and Machine Translation, October 2016. URL <http://arxiv.org/abs/1609.08144>. arXiv:1609.08144 [cs].

- [24] Zi Yun Yang, Ziqing Zhang, and Yisong Miao. The ELCo Dataset: Bridging Emoji and Lexical Composition. In Nicoletta Calzolari, Min-Yen Kan, Veronique Hoste, Alessandro Lenci, Sakriani Sakti, and Nianwen Xue, editors, *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, pages 15899–15909, Torino, Italia, May 2024. ELRA and ICCL. URL <https://aclanthology.org/2024.lrec-main.1381/>.